DES3UE WS 2026 Übung 2

Abgabetermin: siehe e-Learning, Abgabeform elektronisch

☒ **DES3UEG1 (Niklas)** **Name** Klemens Danner **Aufwand in h** 7,5
☐ **DES3UEG2 (Niklas)** **Punkte** _____ **Kurzzeichen Tutor*in** _____
☐ **DES3UEG3 (Teufl)**

Hinweise und Richtlinien:

Die Übungsausarbeitungen müssen den im eLearning angegebenen Formatierungsrichtlinien entsprechen. Nichtbeachtung dieser Formatierungsrichtlinien führt zu Punkteabzug.

Erstellen Sie für die angeführte Fragestellung jeweils eine Abfrage, verwenden Sie als fixe Werte nur jene die auch in der Angabe gegeben sind.

Ziel dieser Übung ist es, die Basiselemente der Anfragesprache SQL mit einem neuen Datenbank-Schema zu wiederholen. Dabei wird das Schema der „Sakila-Datenbank“ für die folgenden Übungen eingeführt.

1. Grundlagen (7 Punkte)

1. Erstellen Sie eine Liste aller Schauspieler*innen. Verketten Sie Vor- und Nachname (getrennt durch ein Leerzeichen) und nennen Sie die neue Spalte „Name“. Sortieren Sie das Ergebnis nach dem Nachnamen. *(1 Punkt)*
2. Geben Sie eine Liste mit Titel und Länge all jener Filme aus, die mindestens 120 Minuten dauern und deren Titel an zweiter Stelle ein 'A' enthält. Geben Sie die Titel so aus, dass jeweils der erste Buchstabe eines Wortes mit einem Großbuchstaben beginnt (zB Calendar Gunfight). *(1 Punkt)*
3. Geben Sie alle Inventar-Ids aus, die noch nie verliehen wurden. Verwenden Sie einen Mengenoperator. *(1 Punkt)* union, intersect etc.
4. Geben Sie für alle Mitarbeiter*innen folgende Informationen aus: den Nachnamen als „StaffName“, die ID (store_id) der zugeordneten Filiale sowie den Nachnamen des Managers der jeweiligen Filiale als „ManagerName“. *(1,5 Punkte)*
5. Geben Sie die Anzahl der Verleihvorgänge im Jahr 2015 aus (Start des Verleihvorgangs, *rental_date*). *(1 Punkt)*
6. Geben Sie für alle Verleihvorgänge die ID sowie das Rückgabedatum (in absteigender Reihenfolge, nicht abgeschlossene Verleihvorgänge zuerst) in folgendem Format aus: ‚*Wed, 14th October 2026*‘. Wenn noch kein Rückgabedatum vorhanden ist, soll stattdessen der Text „not returned yet“ ausgegeben werden. Verwenden Sie zur Formatierung des Datums die Funktion TO_CHAR. Achten Sie darauf, die Wochentags- und Monatsnamen in englischer Sprache auszugeben, indem Sie den passenden NLS-Parameter setzen. Recherchieren Sie bei Bedarf in der Oracle-Dokumentation. *(1,5 Punkte)*

to_char

2. Gruppierungen und Unterabfragen (11 Punkte)

1. Geben Sie Vor- und Nachnamen aller Schauspieler*innen aus, die in keinem Film der Kategorie „Horror“ mitgespielt haben. Verwenden Sie KEINEN Mengenoperator.
   (44 Zeilen, 1,5 Punkte)

2. Geben Sie die Vor- und Nachnamen aller Kund*innen aus, die eine Adresse in den „United States“ aufweisen, deren zugewiesene Hauptfiliale (`customer.store_id`) sich aber in „Canada“ befindet.
   (17 Zeilen, 1,5 Punkte)
3. Geben Sie Vor- und Nachnamen sowie den Gesamtumsatz (Summe aller Zahlungen) jener Kund*innen aus, deren Gesamtumsatz über dem durchschnittlichen Gesamtumsatz pro Kund*in liegt. Sortieren Sie absteigend nach dem Umsatz.
   (285 Zeilen, 2 Punkte)
4. Ermitteln Sie alle Filmtitel, die kürzer oder gleich lange dauern als der Film mit der ID 50. Geben Sie von diesen Titeln allerdings nur jene mit den größten Ersetzungskosten aus. Sortieren Sie nach Filmtitel.
   (53 Zeilen, 2 Punkte)
5. Ermitteln Sie für jede Sprache den längsten Film mit einer Dauer von höchstens 120 Minuten. Verwenden Sie dafür die Spalte `language_id`. Berücksichtigen Sie nur Filme, bei denen eine Originalsprache (`original_language_id`) hinterlegt ist. Haben mehrere Filme dieselbe maximale Dauer, sollen alle ausgegeben werden. Geben Sie den Namen der Sprache sowie die Titel und Länge der Filme aus, sortieren Sie nach Sprache.
   (11 Zeilen, 2,5 Punkte)
6. Geben Sie die neun zuletzt verliehenen Filme und das Verleihdatum im Format ‘YYYY-MM-DD‘ aus. Welche Möglichkeiten haben Sie, um die „neun letzten“ Filme zu ermitteln?
   (1,5 Punkte)

3. Insert, Update und Delete

aufbauend, in dieser Reihenfolge abarbeiten **(6 Punkte)**

1. Erstellen Sie eine neue Tabelle „new_film“, diese soll den gleichen Aufbau wie die Tabelle „film“ haben, jedoch nur die neueren Filme enthalten. Als neuere Filme gelten jene, die nicht älter als 25 Jahre sind (setzen Sie dazu das aktuelle Datum in Bezug mit dem Erscheinungsjahr).
   (1 Punkt)
2. Fügen Sie den Film „Love Machine 3“ mit ID = 1001 in Englisch (*language_id* = 1) mit 5 Tagen Verleihdauer (zum Preis von 1,79) mit Ersetzungskosten von 21,50 in die Tabelle ein.
   (1 Punkt)
3. Erhöhen Sie den Leihpreis der Filme in der Tabelle „new_film“ um 15%, wenn der Leihpreis kleiner als 2 ist. (*0,5 Punkte)*
4. Erstellen Sie eine View, die alle Filme der Tabelle „new_film“ enthält, deren Leihpreis maximal 2 beträgt, vergeben Sie die Check-Option. Die View soll nur den Filmtitel, die Beschreibung, den Leihpreis und die Länge enthalten. *(1 Punkt)*
5. Welche Auswirkungen haben COMMIT und ROLLBACK an dieser Stelle (nachdem Sie die View erstellt haben). (*0,5 Punkte)* nur Frage
6. Können Sie die Datensätze Ihrer neuen View verändern? Wenn ja, führen Sie die Erhöhung der Filme (15%) ein weiteres Mal durch, diesmal auf Ihre View. Wenn nein, warum nicht? (*0,5 Punkte)*
7. Löschen Sie alle Einträge aus der Tabelle „new_film“, deren Leihpreis über 1.79 liegt. (*0,5 Punkte)*
8. Können Sie den Leihpreis der Filme in der View nun erhöhen? Erklären Sie dieses Verhalten. (*0,5 Punkte)*
9. Löschen Sie die Tabelle „new_film“ und Ihre erstellte View wieder. (2 Statements, *0,5 Punkte)*

Aufgabe 1

1.1

Code

```
SELECT first_name || '' || last_name AS Name
FROM actor
ORDER BY last_name;
```

Ausgabe

```
DEBBIE AKROYD
CHRISTIAN AKROYD
KIRSTEN AKROYD
MERYL ALLEN
...
CAMERON ZELLWEGER
```

```
(200 selected)
```

1.2

Code

```
SELECT INITCAP(title), length
FROM film
WHERE length >= 120 AND title LIKE '_A%';
```

Ausgabe

```
Baby Hall
Badman Dawn
Baked Cleopatra
Ballroom Mockingbird
...
Raging Airplane
```

```
(88 selected)
```

1.3

Code

```
SELECT inventory_id
FROM inventory
MINUS
SELECT inventory_id
FROM rental;
```

Ausgabe

```
5
```

```
(1 selected)
```

1.4

Code

```
SELECT employee.last_name AS StaffName,
employee.store_id AS ID,
manager.last_name AS ManagerName
FROM staff employee
JOIN store ON store.store_id = employee.store_id
JOIN staff manager ON store.manager_staff_id = manager.staff_id;
```

Ausgabe

	STAFFNAME	ID	MANAGERNAME
1	Hillyer	1	Hillyer
2	Stephens	2	Stephens
3	Miller	3	Miller
4	Hynf	4	Hynf
5	Brown	5	Brown
6	Delfino	5	Brown
7	Delfino	5	Brown
8	Miller	5	Brown

(8 selected)

1.5

Code

```
SELECT COUNT(*) AS CtRentals
FROM rental
WHERE EXTRACT(YEAR FROM rental_date) = 2015;
```

Ausgabe

	CTRENTALS
1	7050

(selected)

1.6

Code

```
SELECT rental_id, COALESCE(TO_CHAR(return_date, 'FMDy, ddth Month YYYY',
'NLS_DATE_LANGUAGE = english'), 'not returned yet')
FROM rental
ORDER BY return_date DESC;
```

Ausgabe

	RENTAL_ID	RETURN_DATE
1	13862	not returned yet
2	15932	not returned yet
3	14635	not returned yet
4	1353	Wed, 9th September 2015
5	46	Wed, 9th September 2015
6	15856	Wed, 9th September 2015

...

16044	12402	Fri, 10th April 2015

(16044 selected)

Aufgabe 2

2.1

Code

```
SELECT first_name, last_name
FROM actor
WHERE actor_id NOT IN
      (SELECT actor_id
      FROM actor
         JOIN film_actor USING (actor_id)
         JOIN film_category USING (film_id)
         JOIN category USING (category_id)
      WHERE category.name = 'Horror');
```

Ausgabe

	FIRST_NAME	LAST_NAME
1	SCARLETT	BENING
2	NATALIE	HOPKINS
3	BETTE	NICHOLSON
4	NICK	DEGENERES
...		
44	CHARLIZE	DENCH

(44 selected)

2.2

Code

```
SELECT c.first_name, c.last_name
FROM customer c
    -- Kundenpfad
    JOIN address ca ON c.address_id = ca.address_id
    JOIN city cci ON ca.city_id = cci.city_id
    JOIN country cco ON cci.country_id = cco.country_id
    -- store Pfad
    JOIN store cs ON c.store_id = cs.store_id
    JOIN address st_addr ON cs.address_id = st_addr.address_id
    JOIN city st_city ON st_addr.city_id = st_city.city_id
    JOIN country st_country ON st_city.country_id = st_country.country_id
WHERE cco.country = 'United States' AND st_country.country = 'Canada';
```

Ausgabe

	FIRST_NAME	LAST_NAME
1	ZACHARY	HITE
2	EVA	RAMOS
3	JAMIE	RICE
4	VALERIE	BLACK
...		
17	SHELLY	WATTS

(17 selected)

2.3

Code

```
SELECT first_name, last_name, SUM(amount)
FROM customer
      JOIN payment USING (customer_id)
GROUP BY customer_id, first_name, last_name
HAVING SUM(amount) > (
-- durchschnittlicher Gesamtumsatz pro Kunde
            SELECT AVG(SUM(amount))
            FROM customer
            JOIN payment USING (customer_id)
            GROUP BY customer_id)
ORDER BY SUM(amount) DESC;
```

Ausgabe

	FIRST_NAME	LAST_NAME	SUM(AMOUNT)
1	MARCIA	DEAN	357.39
2	JUNE	CARROLL	356.74
3	WESLEY	BULL	342.67
4	MARSHA	DOUGLAS	333.69

...

285	LEO	EBERT	193.26

(285 selected)

2.4

Code

```
SELECT title
FROM film
WHERE length <= (SELECT length FROM film WHERE film_id = 50)
      AND replacement_cost = (
            SELECT MAX(replacement_cost)
            FROM film
            WHERE length <= (SELECT length FROM film WHERE film_id = 50)
            )
ORDER BY title;
```

Ausgabe

	TITLE
1	ARABIA DOGMA
2	BALLROOM MOCKINGBIRD
3	BLINDNESS GUN

...

53	WYOMING STORM

(53 selected)

2.5

Code

```
SELECT l.name, f.title, f.length
FROM film f
      JOIN language l ON f.language_id = l.language_id
WHERE f.original_language_id IS NOT NULL
      AND f.length <= 120
      AND f.length = (
            SELECT MAX(f1.length)
            FROM film f1
            WHERE f1.language_id = f.language_id
            AND f1.length <= 120
            AND f1.original_language_id IS NOT NULL
            )
ORDER BY l.name;
```

Ausgabe

	NAME	TITLE	LENGTH
1	English	GRAFFITI LOVE	117
2	French	COMMAND DARLING	120
3	German	DOLLS RAGE	120
4	German	ORDER BETRAYED	120
5	German	LOCK REAR	120
6	Italian	RAGE GAMES	120
7	Italian	UNTOUCHABLES SUNRISE	120
8	Japanese	DAZED PUNK	120
9	Japanese	CALENDAR GUNFIGHT	120
10	Japanese	KARATE MOON	120
11	Mandarin	GAMES BOWFINGER	119

(11 selected)

2.6

Code

```
SELECT title, TO_CHAR(rental_date, 'YYYY-MM-DD') AS rental_date
FROM film
      JOIN inventory USING (film_id)
      JOIN rental USING (inventory_id)
ORDER BY rental_date DESC
FETCH FIRST 9 ROWS ONLY;
```

Ausgabe

	TITLE	RENTAL_DATE
1	SOMETHING DUCK	2015-11-04
2	GOODFELLAS SALUTE	2015-11-04
3	ATLANTIS CAUSE	2015-11-04
4	ANONYMOUS HUMAN	2015-11-04
5	YENTL IDAHO	2015-11-04
6	VELVET TERMINATOR	2015-11-04
7	VIRTUAL SPOILERS	2015-11-04
8	SWEETHEARTS SUSPECTS	2015-11-04
9	ENCOUNTERS CURTAIN	2015-11-04

(9 selected)

Aufgabe 3

3.1

Code

```
CREATE TABLE new_film AS
SELECT *
FROM film
WHERE EXTRACT(YEAR FROM CURRENT_DATE) - release_year <= 25;
```

Ausgabe

```
Table NEW_FILM created.
```

3.2

Code

```
INSERT INTO new_film (title, film_id, language_id, rental_duration,
rental_rate, replacement_cost, last_update)
VALUES ('Love Machine 3', 1001, 1, 5, 1.79, 21.50, SYSDATE);
```

Ausgabe

```
1 row inserted.
```

3.3

Code

```
UPDATE new_film
SET rental_rate = rental_rate * 1.15
WHERE rental_rate < 2;
```

Ausgabe

```
196 rows updated.
```

3.4

Code

```
CREATE VIEW cheap_new_film AS
SELECT title, description, rental_rate, length
FROM new_film
WHERE rental_rate <= 2
WITH CHECK OPTION;
```

Ausgabe

```
View CHEAP_NEW_FILM created.
```

3.5

Welche Auswirkungen haben COMMIT und ROLLBACK an dieser Stelle (nachdem Sie die View erstellt haben).

Die CREATE VIEW Anweisung hat implizit einen COMMIT abgesetzt, weil selbst nach einem weiteren ROLLBACK werden jetzt beim Statement aus Aufgabe 3.3 nur mehr "171 rows updated.", anstatt der vorherigen "196 rows updated.". Die Änderung von Aufgabe 3.3 wurde also entgültig in die Datenbank übernommen.
ROLLBACK setzt also nur bis zum letzten "COMMIT" zurück - in diesem Fall also nur bis zum CREATE VIEW statement.

3.6

```
/* UPDATE cheap_new_film
SET rental_rate = rental_rate * 1.15; */
```

Das darf nicht funktionieren, weil durch die Operation Datensätze aus der VIEW herausfallen würden - das wird aber durch die CHECK OPTION von der VIEW verhindert.

```
Error starting at line : 1 in command -
UPDATE cheap_new_film
SET rental_rate = rental_rate * 1.15
Error at Command Line : 1 Column : 8
Error report -
SQL Error: ORA-01402: view WITH CHECK OPTION where-clause violation
Help: https://docs.oracle.com/error-help/db/ora-01402/
```

3.7

Code

```
DELETE FROM new_film
WHERE rental_rate > 1.79;
```

Ausgabe

```
157 rows deleted.
```

3.8 Kann man die Sicht jetzt verändern?

Ja, das geht, weil alle Werte gelöscht wurden, welche bei Multiplikation mit 1.15 größer als 2 sein würden und somit aus der View herausfallen würden. Daher lässt die View das zu, weil keine Werte aus dem "Sichtfeld:)" verschwinden würden.

Code

```
UPDATE cheap_new_film
SET rental_rate = rental_rate * 1.15;
```

Ausgabe

```
152 rows updated.
```

3.9

Code

```
DROP VIEW cheap_new_film;
DROP TABLE new_film;
```

Ausgabe

```
View CHEAP_NEW_FILM dropped.


Table NEW_FILM dropped.
```